
Labwork 2.2-4 - Calendar Application

General Information and Submission Rules:

- In this labwork, you need to design a calendar application that does different operations.
- By the end of the labwork, you must submit it to Yulearn as **studentID_fullname_labwork2.c**.
- Incorrect naming or file format will result in a loss of 5 points from your total grade.
- Only use concepts covered in the labs so far.

Question 1 - Calculate Meeting Number (25p)

- Create a function named (**meeting_calculator**) that calculates the total number of meetings a person has attended in a given month, up to a specific day.
- The number of meetings held on any day is exactly equal to the date itself.
- Furthermore, the schedule depends on whether the month is even or odd:
 - # In even-numbered months, meetings only occur on even-numbered days.
 - # In odd-numbered months, meetings only occur on odd-numbered days.
- For instance,
 - # If the month is 4 (even) and the day is 4, meetings occur on days 2 and 4. The total is $2+4=6$ meetings.
 - # If the month is 9 (odd) and the day is 7, meetings occur on days 1, 3, 5 and 7. The total is $1+3+5+7=16$.
- You need to use a given summation formulas based on the type of the month to calculate the total meeting number:

$$evenMeeting = n(n + 1)$$

$$oddMeeting = n^2$$

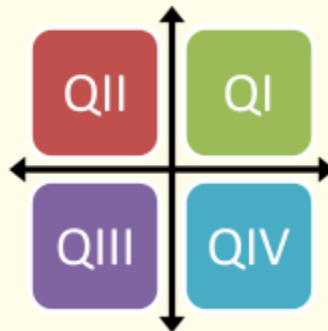
- The function should take the month and the day as inputs from the user and return the total number of meetings.

```
>> Enter the month number: 4
>> Enter the day: 4
The total meeting number is 6.

>> Enter the month number: 9
>> Enter the day: 7
The total meeting number is 16.
```

Question 2 - Calendar Region Selection (50p) and Promotion Detection (25p)

- Write a function named (**detect_region**) that calculates an employee's regional performance score based on their location data.
- A month is divided into four distinct regions, and we track an employee's location on one specific day within each of these regions.
- The function should take the (x, y) coordinate of the employee's location for 3 days as user input.
- Each of the regions has a predefined coefficient.
- To find the total score, multiply each day's x-coordinate by its corresponding coefficient and sum the results.



Q1	Q2	Q3	Q4
5	1	2	3

- Write another function named (**compare_two**) that compares two candidates to determine who receives a promotion.
- Repeat this process for two people (Person A and Person B).
- The function should take the performance scores of both individuals as inputs.
- In the end, it must compare them and detect the person with the higher total score. This person will receive the promotion.
- The all input and output format is given below.

```
>> Enter first location: 4 5
>> Enter second location: -4 -5
>> Enter third location: 3 -7
First: Q1
Second: Q3
Third: Q4
Total score is 21.

>> Enter first location: -2 -5
>> Enter second location: -4 -6
>> Enter third location: 10 -5
First: Q3
Second: Q3
Third: Q4
Total score is 18.

Selected person: A
```